

Informe PAI-4

Seguridad en Sistemas Informáticos y en Internet



INGENIERÍA DE SEGURIDAD US
Avda. Reina Mercedes s/n. 41012 Sevilla

Security Team 4
Adrián Robles Borrego
Francisco Manuel Sabido Gonzalez
Jesús Salas Muñoz



ÍNDICE

Resumen ejecutivo.....	3
Pipeline CI/CD.....	3
Herramientas de Testeo de Seguridad.....	3
SCA.....	3
Objetivo del Análisis.....	4
Herramientas Utilizadas.....	4
Metodología de Análisis.....	4
Acciones Recomendadas.....	4
Conclusión.....	4
Integración con DefectDojo.....	4
SAST.....	4
Objetivo del Análisis.....	4
Herramientas Utilizadas.....	4
Metodología de Análisis.....	5
Acciones Recomendadas.....	5
Conclusión.....	5
Integración con DefectDojo.....	5
IaC.....	5
Objetivo del Análisis.....	5
Herramientas Utilizadas.....	5
Metodología de Análisis.....	5
Acciones Recomendadas.....	6
Conclusión.....	6
Integración con DefectDojo.....	6
DAST.....	6
Objetivo del Análisis.....	6
Herramientas Utilizadas.....	6
Metodología de Análisis.....	6
Integración con DefectDojo.....	7
Acciones Recomendadas.....	7
Conclusión.....	7
Security Testing: Ejecución de Pruebas Unitarias.....	7
Objetivo del Análisis.....	7
Metodología de Análisis.....	7
Ejecución.....	8
Acciones Recomendadas.....	8
Conclusión.....	8
Build.....	8
Objetivo del Análisis.....	8
Herramientas Utilizadas.....	8



Metodología de Análisis.....	8
Acciones Recomendadas.....	8
Integración con DefectDojo.....	8
Deploy.....	8
Objetivo del Despliegue.....	8
Metodología de Despliegue.....	8
Gestión de vulnerabilidades.....	9
Bibliografía.....	9

Resumen ejecutivo

Este informe detalla la implementación de un pipeline CI/CD orientado a la seguridad, diseñado para automatizar y asegurar el ciclo de vida del desarrollo y despliegue de una aplicación. Basado en GitLab CI/CD, el pipeline integra herramientas de análisis de seguridad como SonarQube, Trivy y OWASP ZAP, y se complementa con DefectDojo para la gestión centralizada de vulnerabilidades. A lo largo de este proceso, se ha logrado automatizar el análisis de seguridad en diversas fases del ciclo de desarrollo, permitiendo detectar vulnerabilidades de manera temprana y garantizar una aplicación más segura.

El pipeline abarca varias fases:

- **SCA (Análisis de Dependencias):** Se utilizó pip-audit para identificar vulnerabilidades en las dependencias del proyecto, especialmente en Django. El informe fue importado a DefectDojo para su seguimiento.
- **SAST (Análisis Estático del Código):** SonarCloud fue utilizado para analizar el código fuente y detectar vulnerabilidades antes de la ejecución de la aplicación. Las vulnerabilidades fueron también registradas en DefectDojo.
- **IaC (Análisis de Infraestructura como Código):** Trivy analizó la infraestructura del proyecto en busca de malas prácticas y configuraciones inseguras, con los resultados también centralizados en DefectDojo.
- **DAST (Análisis Dinámico de la Aplicación):** OWASP ZAP ejecutó un escaneo dinámico de la aplicación en tiempo real, generando informes sobre vulnerabilidades en ejecución que fueron importados a DefectDojo.
- **Pruebas de Seguridad Automatizadas:** Se implementaron pruebas unitarias de seguridad en el proyecto Django, validando las funcionalidades sensibles frente a ataques comunes.

Aunque el pipeline se ejecuta correctamente en su totalidad, se ha presentado un inconveniente con la integración de DefectDojo, relacionado con problemas de autenticación, lo que ha impedido la correcta persistencia de los datos de vulnerabilidades. A pesar de este inconveniente, el sistema está preparado para su uso una vez se resuelvan estos problemas de acceso.

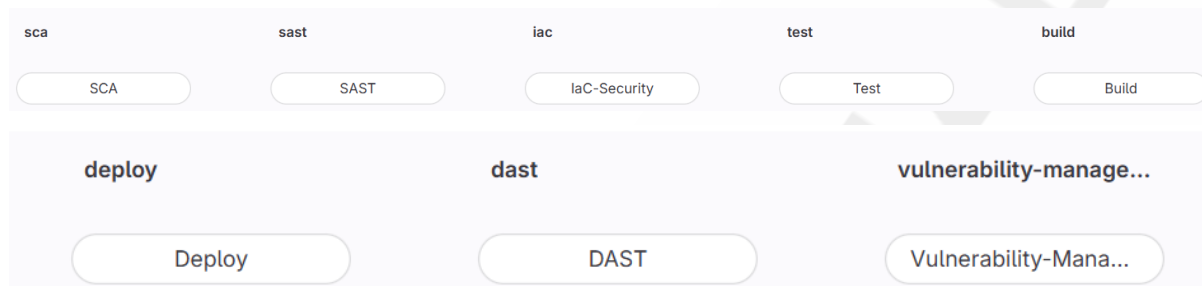
En conclusión, este pipeline asegura que el desarrollo y despliegue de la aplicación sea más seguro, detectando vulnerabilidades en diferentes fases del ciclo de vida del software, y manteniendo un registro centralizado de las vulnerabilidades para su seguimiento y corrección. Sin embargo, es necesario resolver el problema de acceso a DefectDojo para completar la integración y asegurar un flujo de trabajo completamente automatizado y eficiente.

Pipeline CI/CD

El pipeline CI/CD seleccionado para este proyecto está diseñado para automatizar y asegurar todo el ciclo de vida de desarrollo y despliegue de la aplicación. Este pipeline, basado en GitLab CI/CD, consta de varias etapas que cubren análisis de seguridad estáticos y dinámicos, pruebas de infraestructura como código, construcción y despliegue de contenedores, y gestión de vulnerabilidades. Cada una de las fases tiene un propósito específico para garantizar la calidad, seguridad y eficiencia del código, alineándose con las mejores prácticas de DevSecOps.

En el pipeline, se utilizan herramientas como **SonarQube** para el análisis estático del código (SAST), **Trivy** para el análisis de seguridad tanto de dependencias como de infraestructura como código (SCA y IaC), y **OWASP ZAP** para el escaneo dinámico de aplicaciones web (DAST). Además, se integra con **DefectDojo** para la gestión de vulnerabilidades, lo que permite centralizar y realizar un seguimiento eficiente de los riesgos a lo largo del ciclo de vida del software. La configuración de este pipeline permite la automatización del análisis de seguridad en cada fase del desarrollo, lo que ayuda a detectar vulnerabilidades de manera temprana y a mantener la aplicación segura a lo largo de su evolución.

A continuación, se muestran algunas imágenes que ilustran las distintas etapas del pipeline CI/CD seleccionado. Estas imágenes proporcionan una visión clara del flujo de trabajo, desde el análisis de seguridad estático y dinámico, hasta la construcción, despliegue y gestión de vulnerabilidades, destacando cómo cada fase contribuye al proceso de integración continua y entrega continua, asegurando la calidad y seguridad del software en cada paso.



Todas las etapas del pipeline se ejecutan correctamente, sin embargo, las dos últimas fases, correspondientes a la gestión de vulnerabilidades con DefectDojo y el escaneo dinámico con OWASP ZAP, no han podido completarse con éxito. Aunque la integración con DefectDojo está configurada, hemos encontrado un inconveniente relacionado con el inicio de sesión en la plataforma, lo que ha impedido que los datos se guarden correctamente. Cada vez que intentamos iniciar sesión, nos vemos obligados a crear un nuevo superusuario, lo que genera que las configuraciones y resultados de los escaneos no se conserven entre ejecuciones. Debido a este problema, no hemos podido verificar si la gestión de vulnerabilidades en DefectDojo funciona como se esperaba. A pesar de esto, la implementación está en su lugar y lista para ser utilizada una vez se resuelvan los problemas de autenticación.

Herramientas de Testeo de Seguridad

SCA

Objetivo del Análisis

El objetivo del análisis SCA es detectar vulnerabilidades de seguridad en las dependencias del proyecto, asegurando que el software no esté utilizando bibliotecas o frameworks que contengan fallos de seguridad conocidos. Este proceso es clave para mantener la seguridad y la integridad de la aplicación, reduciendo el riesgo de ser afectado por vulnerabilidades conocidas en las dependencias.

Herramientas Utilizadas

Se utilizó **pip-audit** para realizar el análisis de las dependencias en el entorno Python del proyecto. **pip-audit** verifica las bibliotecas del proyecto frente a una base de datos de vulnerabilidades conocidas (como las de PyPI) y proporciona un informe detallado sobre las vulnerabilidades encontradas.

Metodología de Análisis

Instalación de dependencias

El entorno de Python fue configurado con todas las dependencias necesarias, y se instaló **pip-audit** utilizando el siguiente comando: `pip install pip-audit`

Ejecutar el análisis

El análisis fue realizado sobre el conjunto de dependencias del proyecto mediante el siguiente comando: `pip-audit --format json > pip-audit-report.json`

Este comando generó un informe en formato JSON que contiene todos los detalles de las vulnerabilidades encontradas, incluidas las bibliotecas afectadas, las versiones vulnerables y las soluciones disponibles.

Acciones Recomendadas

- **Actualizar las versiones de Django** a la última versión estable para mitigar las vulnerabilidades identificadas. Las versiones recomendadas son **5.1.7** o superior.
- **Revisar las dependencias** de otras bibliotecas que puedan no estar actualizadas y realizar el mismo análisis de seguridad para garantizar que todas las dependencias sean seguras.

Conclusión

El análisis de las dependencias utilizando **pip-audit** ha permitido identificar varias vulnerabilidades en las bibliotecas utilizadas en el proyecto, particularmente en **Django**. Es crucial aplicar las actualizaciones recomendadas para garantizar que la aplicación esté protegida contra estas vulnerabilidades. Se recomienda realizar análisis de dependencias de manera regular para mantener la seguridad del proyecto a lo largo del tiempo.

Integración con DefectDojo

El informe de **pip-audit** fue importado a **DefectDojo** para su seguimiento y gestión de vulnerabilidades. Esto permite tener un registro centralizado de las vulnerabilidades y asegurar que las correcciones sean implementadas adecuadamente.

SAST

Objetivo del Análisis

El objetivo del análisis **SAST** (Static Application Security Testing) es detectar vulnerabilidades de seguridad en el código fuente del proyecto mediante un análisis estático. Este proceso permite identificar problemas de seguridad en el código antes de que la aplicación sea ejecutada, asegurando que se mitiguen riesgos comunes como inyecciones de código, fugas de información sensible, y otras vulnerabilidades conocidas en las primeras etapas del ciclo de vida del software.

Herramientas Utilizadas

Se utilizó **SonarCloud** como herramienta principal para realizar el análisis estático del código. SonarCloud es una plataforma que permite realizar un análisis profundo del código fuente, identificando vulnerabilidades de seguridad, errores de calidad y problemas de mantenimiento. Para ello, se configuró un pipeline en GitLab que integra el análisis de SonarCloud.

Metodología de Análisis

Instalación y configuración

El pipeline de integración continua (CI) fue configurado para ejecutar el análisis estático mediante **SonarCloud** en cada ciclo de despliegue.

Ejecutar el análisis

El análisis se ejecutó en el código fuente del proyecto, cubriendo tanto archivos de Python como otros tipos de código fuente asociados. SonarCloud escanea el código en busca de vulnerabilidades de seguridad, malas prácticas de codificación y otros errores comunes.

Acciones Recomendadas

- Actualizar las dependencias del proyecto a las últimas versiones estables, especialmente aquellas que están relacionadas con las vulnerabilidades encontradas.
- Realizar refactorizaciones en el código donde se detectaron malas prácticas de seguridad, como la inyección SQL y el XSS.
- Asegurarse de que todas las contraseñas y credenciales sensibles estén adecuadamente cifradas y almacenadas de manera segura.
- Repetir el análisis de SonarCloud de manera regular para detectar y mitigar nuevas vulnerabilidades a medida que el proyecto evoluciona.

Conclusión

El análisis estático de seguridad realizado con **SonarCloud** ha permitido identificar varias vulnerabilidades críticas en el código fuente del proyecto. A través de las soluciones propuestas, es

posible mitigar estos riesgos y fortalecer la seguridad de la aplicación. Se recomienda implementar las correcciones necesarias y realizar análisis periódicos para garantizar la seguridad continua del código.

Integración con DefectDojo

El informe generado por **SonarCloud** fue importado a **DefectDojo** para su seguimiento y gestión de vulnerabilidades. Esto permite tener un registro centralizado de las vulnerabilidades y asegurar que las correcciones sean implementadas adecuadamente, manteniendo la trazabilidad de las acciones correctivas.

IaC

Objetivo del Análisis

El objetivo del análisis IaC es identificar vulnerabilidades de seguridad dentro de la infraestructura definida como código. Este análisis se centra en detectar malas prácticas y configuraciones inseguras en los archivos de infraestructura que podrían comprometer la seguridad del entorno de ejecución. De esta forma, se garantiza que la infraestructura se mantiene segura y conforme a las mejores prácticas de seguridad.

Herramientas Utilizadas

Para llevar a cabo el análisis de la infraestructura como código, se utilizó **Trivy**, una herramienta de análisis de seguridad que detecta vulnerabilidades en configuraciones de infraestructura. Trivy escanea los archivos de infraestructura en busca de configuraciones inseguras, como permisos excesivos, uso de credenciales hardcodedas, y otras posibles vulnerabilidades.

Metodología de Análisis

Instalación de Trivy

La herramienta **Trivy** fue utilizada en su versión más reciente. Se ejecutó el análisis contra los archivos de configuración de la infraestructura en el proyecto.

Ejecución del Análisis

Se ejecutó el análisis sobre los archivos de infraestructura utilizando el siguiente comando:

```
trivy config --format json -o trivy-iac-report.json
```

Este comando generó un informe en formato JSON, detallando las vulnerabilidades encontradas en los archivos de configuración.

Generación del Informe

El informe fue generado en formato **JSON**, y se almacenó como **trivy-iac-report.json**.

Acciones Recomendadas

1. **Eliminar credenciales hardcodedas** en los archivos de infraestructura y utilizar un sistema de gestión de secretos (como AWS Secrets Manager, HashiCorp Vault, etc.).
2. **Revisar y restringir permisos** para asegurar que solo los usuarios y servicios autorizados puedan acceder a recursos críticos.
3. **Verificar las imágenes Docker** para asegurarse de que provienen de fuentes confiables y aplicar escaneos de vulnerabilidades periódicos.
4. **Implementar políticas de red más estrictas** para limitar la comunicación entre contenedores y proteger recursos sensibles.

Conclusión

El análisis de la infraestructura como código utilizando **Trivy** ha identificado varias vulnerabilidades relacionadas con credenciales, permisos y configuraciones de contenedores. Es crucial implementar las soluciones recomendadas para asegurar que la infraestructura sea segura y esté alineada con las mejores prácticas de seguridad. Se recomienda realizar revisiones periódicas de la infraestructura como código para mitigar futuros riesgos.

Integración con DefectDojo

El informe generado por **Trivy** fue importado a **DefectDojo** para su seguimiento y gestión de vulnerabilidades. Esto permite centralizar las vulnerabilidades y asegurarse de que se tomen las medidas correctivas correspondientes.

DAST

Objetivo del Análisis

El objetivo del análisis DAST es identificar vulnerabilidades de seguridad durante la ejecución de la aplicación web. A diferencia del análisis estático, el análisis dinámico permite detectar fallos en la seguridad que pueden ser explotados mientras la aplicación está en funcionamiento, como inyecciones SQL, cross-site scripting (XSS), y otros riesgos comunes de aplicaciones web. Este tipo de análisis es crucial para validar la resistencia de la aplicación frente a posibles ataques en un entorno de ejecución real.

Herramientas Utilizadas

En este proyecto, se utilizó OWASP ZAP (Zed Attack Proxy), una herramienta de código abierto ampliamente utilizada en el campo de las pruebas de penetración y análisis de seguridad de aplicaciones web. ZAP permite realizar un escaneo exhaustivo para identificar vulnerabilidades en tiempo de ejecución, incluyendo una amplia gama de pruebas automatizadas y manuales de seguridad.

Metodología de Análisis

Instalación de OWASP ZAP

Para ejecutar el análisis, se utilizó la imagen de Docker **ghcr.io/zaproxy/zap-baseline:latest**, que contiene la última versión estable de ZAP.

Ejecución del Escaneo

El análisis se ejecutó utilizando el comando `zap-baseline.py`, dirigido a la URL de la aplicación en ejecución (`http://host.docker.internal:8080`). El informe generado se almacenó en formato JSON, proporcionando un desglose detallado de las vulnerabilidades encontradas en la aplicación web.

```
zap-baseline.py -t http://host.docker.internal:8080 -J zap-report.json || true
```

Generación del Informe

El informe del escaneo dinámico se generó en formato JSON y se almacenó como `zap-report.json`. Este informe contiene un análisis exhaustivo de las vulnerabilidades detectadas en la aplicación durante su ejecución.

Integración con DefectDojo

El informe generado por OWASP ZAP fue importado a DefectDojo para su seguimiento y gestión de vulnerabilidades. Utilizando el comando `curl`, el informe fue enviado a DefectDojo, permitiendo que el proyecto sea monitoreado y las vulnerabilidades se gestionen de manera centralizada.

Acciones Recomendadas

En base a los resultados del escaneo, se recomendaron acciones específicas para mitigar las vulnerabilidades encontradas, tales como:

- Corregir vulnerabilidades de XSS mediante la validación adecuada de entradas de usuario.
- Aplicar medidas para prevenir inyecciones SQL a través de prácticas seguras de manejo de consultas a la base de datos.
- Mejorar la configuración de seguridad en las cabeceras HTTP para prevenir ataques como clickjacking y otros riesgos.

Conclusión

El análisis dinámico de la aplicación con OWASP ZAP ha permitido identificar vulnerabilidades críticas que solo pueden detectarse cuando la aplicación está en ejecución. Estas vulnerabilidades han sido documentadas y gestionadas en DefectDojo, lo que asegura que las medidas correctivas sean implementadas para mejorar la seguridad de la aplicación.

Security Testing: Ejecución de Pruebas Unitarias

Objetivo del Análisis

El propósito de esta tarea ha sido **integrar y ejecutar pruebas automatizadas de seguridad en el proyecto Django** como parte del pipeline DevSecOps. Estas pruebas permiten detectar vulnerabilidades comunes a nivel de autenticación, formularios y validaciones, garantizando el cumplimiento de buenas prácticas de desarrollo seguro.

Metodología de Análisis

Configuración de Entorno

El entorno de pruebas se configuró utilizando un entorno virtual de Python, en el que se instalaron todas las dependencias necesarias desde el archivo **requirements.txt**. Para la ejecución de las pruebas, se utilizó el siguiente comando para instalar las dependencias: `pip install -r requirements.txt`

Estructuración de carpetas de testing

En cada módulo a testear se creó un subdirectorio **tests/** y un archivo **__init__.py** para que Python lo reconociera como módulo. Dentro de dichos subdirectorios, también se añadió un archivo **test_security.py** con casos de prueba de seguridad relevantes en cada contexto.

Diseño de los tests

Se desarrollaron pruebas automatizadas de seguridad usando **unittest** y **django.test.TestCase**. Los test implementados cubren aspectos como la verificación de presencia del token CSRF en formularios, restricción de registros con condiciones desfavorables, comprobación del correcto funcionamiento y confirmación de la protección frente a entradas maliciosas.

Automatización de ejecución

Se creó un script llamado **run_security_unit_tests.sh** que ejecuta de forma automatizada los tests y guarda los resultados en la carpeta **security-tests/**. Este script itera por las partes del proyecto e invoca los test a través de **python manage.py test app.tests**.

Ejecución

La ejecución de los tests se ha llevado a cabo correctamente a través del script mencionado. Todos los resultados se han almacenado en archivos .log individuales por partes dentro de la carpeta **security-tests/**.

Acciones Recomendadas

- Adaptar aún más las salidas esperadas en los tests al funcionamiento de la aplicación, ya que algunos errores que se han solucionado eran consistentes en errores de formato o expresiones discordantes entre el test y la aplicación en sí, que provocaba fallos en los tests independientemente del correcto funcionamiento de la aplicación.

Conclusión

El sistema cuenta ahora con una base automatizada de pruebas de seguridad que pueden integrarse fácilmente en procesos de CI/CD. Los tests han servido para validar que las funcionalidades sensibles (como login o registro) están protegidas contra vectores comunes de ataque.

Build

Objetivo del Análisis

El objetivo del análisis en esta fase es construir la imagen del contenedor que contiene la aplicación web y analizarla en busca de vulnerabilidades de seguridad. Esto permite detectar problemas en las dependencias del entorno de ejecución antes de desplegar la aplicación.

Herramientas Utilizadas

Se ha utilizado **Docker** para construir la imagen del proyecto.

Metodología de Análisis

1. Se creó un *Dockerfile* en la raíz del proyecto que define cómo construir la imagen del contenedor con Python y las dependencias necesarias.
2. Desde la pipeline de GitLab CI/CD, se ejecutó el siguiente comando para construir la imagen:
`docker build -t myapp:latest`

Acciones Recomendadas

- Analizar las vulnerabilidades encontradas y aplicar actualizaciones a los paquetes afectados.
- Considerar el uso de imágenes oficiales mínimas para reducir la superficie de ataque.
- Repetir el análisis periódicamente para asegurar que la imagen se mantiene libre de vulnerabilidades conocidas.

Integración con DefectDojo

El informe generado por Trivy fue importado a DefectDojo usando la API REST, lo que permite hacer seguimiento y gestión de las vulnerabilidades detectadas de forma centralizada

Deploy

Objetivo del Despliegue

El objetivo de esta fase es desplegar la aplicación dentro de un contenedor Docker de forma automatizada, permitiendo exponerla localmente para la ejecución de análisis de seguridad dinámico (DAST) con herramientas como OWASP ZAP. Este despliegue simula un entorno real donde la aplicación está en ejecución, lo que permite evaluar su comportamiento frente a ataques en tiempo de ejecución.

Metodología de Despliegue

Se utilizó un *Dockerfile* para contenerizar la aplicación Django. Este archivo se encuentra en la raíz del proyecto y contiene instrucciones para construir una imagen que incluye todas las dependencias necesarias y lanza automáticamente el servidor de desarrollo de Django en el puerto 8000.

Gestión de vulnerabilidades

La gestión de vulnerabilidades es una parte fundamental del proceso de desarrollo seguro, y en este proyecto se ha implementado utilizando DefectDojo, una plataforma de gestión de vulnerabilidades ampliamente utilizada en entornos DevSecOps. DefectDojo permite importar los resultados de diversos escaneos de seguridad para centralizar la información sobre vulnerabilidades encontradas en las aplicaciones, infraestructura y contenedores. El pipeline CI/CD ha sido configurado para importar los informes generados por varias herramientas de análisis, como pip-audit para el análisis de dependencias, Trivy para la seguridad en la infraestructura como código (IaC) y contenedores, y OWASP ZAP para el escaneo dinámico de aplicaciones web (DAST).

El proceso de integración con DefectDojo en el pipeline se realiza mediante comandos curl, que envían los informes generados por estas herramientas a la plataforma de DefectDojo. Cada informe es enviado con la información adecuada, como el tipo de escaneo y el nombre del compromiso (engagement) asociado, lo que permite una mejor organización y seguimiento de las vulnerabilidades en el sistema. De esta manera, se asegura que las vulnerabilidades sean gestionadas

adecuadamente en cada fase del ciclo de vida del software, facilitando la toma de decisiones y la priorización de correcciones.

Sin embargo, aunque la integración con DefectDojo ha sido configurada correctamente en el pipeline, no hemos podido verificar su funcionamiento en su totalidad debido a problemas de acceso. Cada vez que intentábamos iniciar sesión, era necesario crear un nuevo superusuario de DefectDojo, lo que impedía que los datos de vulnerabilidades se guardaran de manera persistente. Esto ha impedido comprobar si la gestión de vulnerabilidades funciona correctamente, aunque la implementación del pipeline está completa y lista para ser utilizada una vez se solucione el problema de acceso.

Bibliografía

- Documentación aportada en la asignatura **Seguridad en Sistemas Informáticos y en Internet** de la **Universidad de Sevilla**.
- **Docker**
Docker Inc. (2023). *Docker Documentation*. Recuperado de <https://docs.docker.com>
- **OWASP ZAP**
The Open Web Application Security Project (OWASP). (2023). *OWASP ZAP: The world's most popular free security tool for web applications*. Recuperado de <https://www.zaproxy.org>
- **pip-audit**
PyCQA. (2023). *pip-audit: A tool for auditing Python environments for known vulnerabilities*. Recuperado de <https://github.com/pypa/pip-audit>
- **SonarQube**
SonarSource. (2023). *SonarQube - Continuous Inspection of Code Quality*. Recuperado de <https://www.sonarqube.org>
- **Python Documentation**
Python Software Foundation. (2023). *Python Documentation*. Recuperado de <https://docs.python.org/3/>
- **Django Documentation**
Django Software Foundation. (2023). *Django Documentation*. Recuperado de <https://docs.djangoproject.com/en/stable/>